

CompSci 234/NetSys 210
Advanced Topics in Networking
Winter 2019

Unit 9: Peer-to-Peer Networks

1

Cheng-Hsin Hsu (chsu@cs.nthu.edu.tw)

Slide adopted from Prof. Venkatasubramanian's and Kurose/Ross' book materials

Agenda

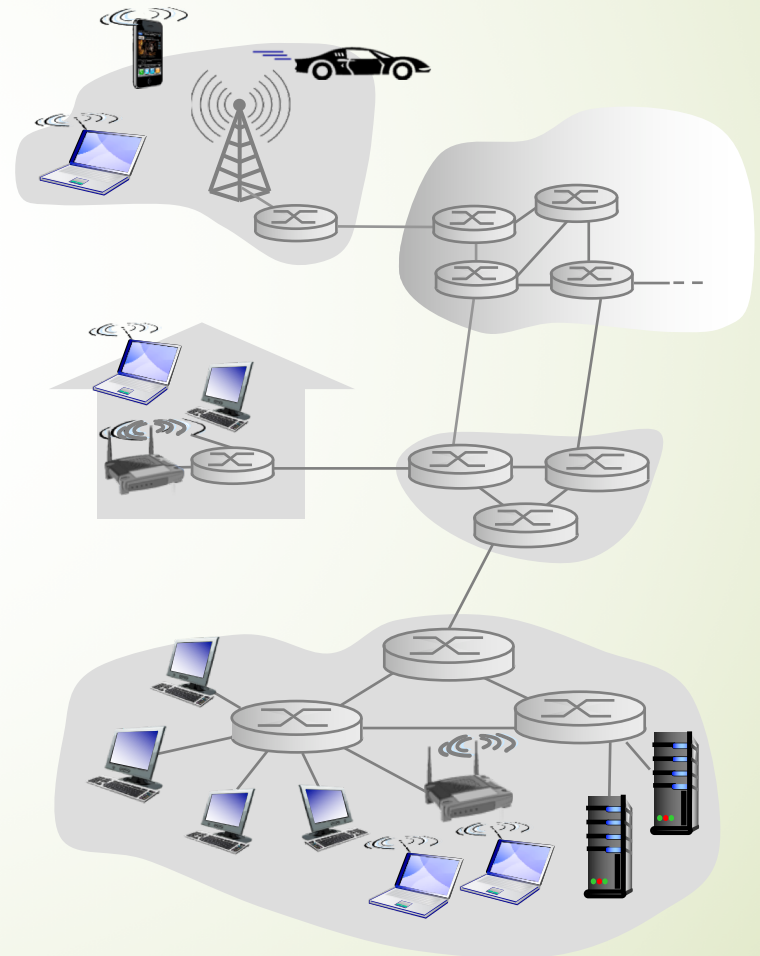
- Overview on P2P Networks
- P2P Applications
- Properties of P2P Systems
- Structured and Unstructured P2P
- Distributed Hash Table (DHT)

Pure P2P Architectures

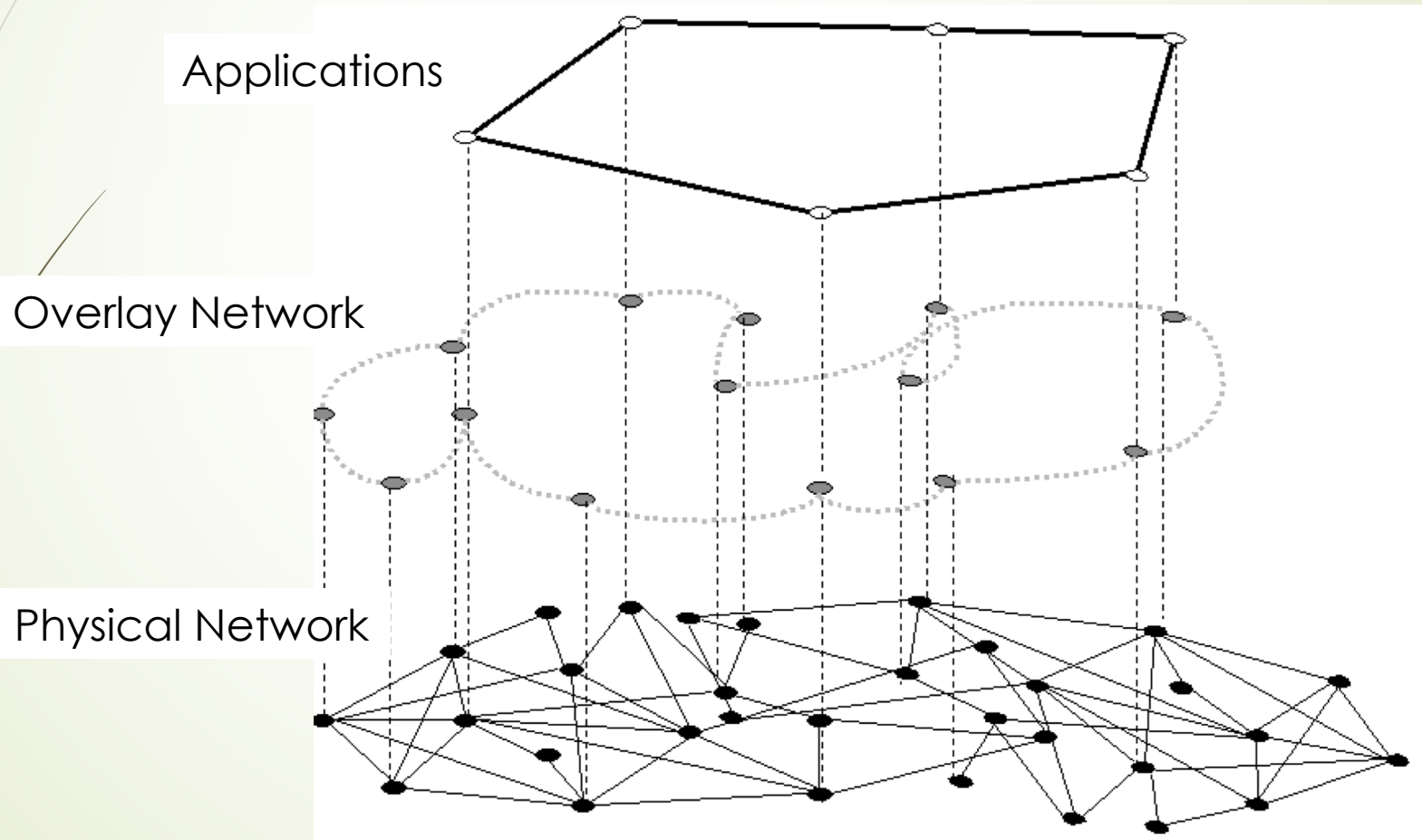
- no always-on server
- arbitrary end systems directly communicate
- peers are intermittently connected and change IP addresses

examples:

- file distribution (BitTorrent)
- Streaming (Xunlei Kankan)
- VoIP (Skype)



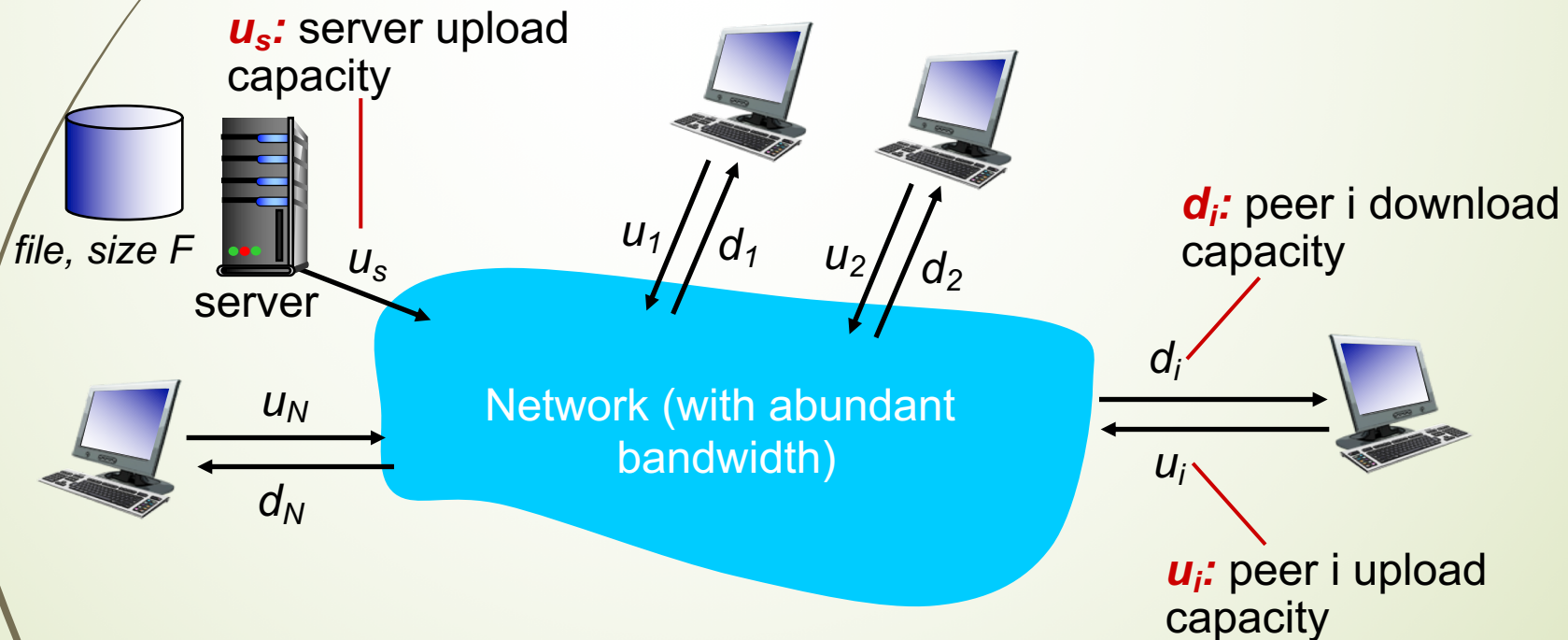
What is the Overlay Network?



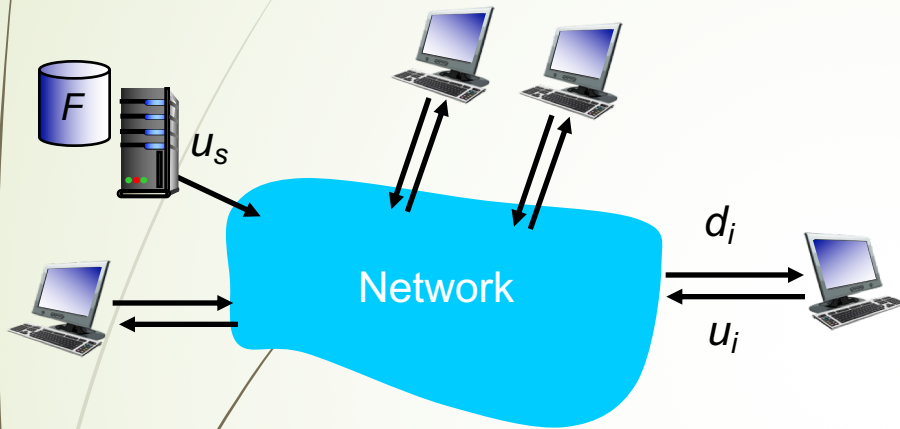
File Dissemination Under Different Architectures

Question: how much time to distribute file (size F) from one server to N peers?

► peer upload/download capacity is limited resource



File Dissemination with Client-Server



- **server transmission:** must sequentially send (upload) N file copies:
 - time to send one copy: F/u_s
 - time to send N copies: NF/u_s
- **client:** each client must download file copy
 - $d_{min} = \min \text{ client download rate}$
 - $\text{min client download time: } F/d_{min}$

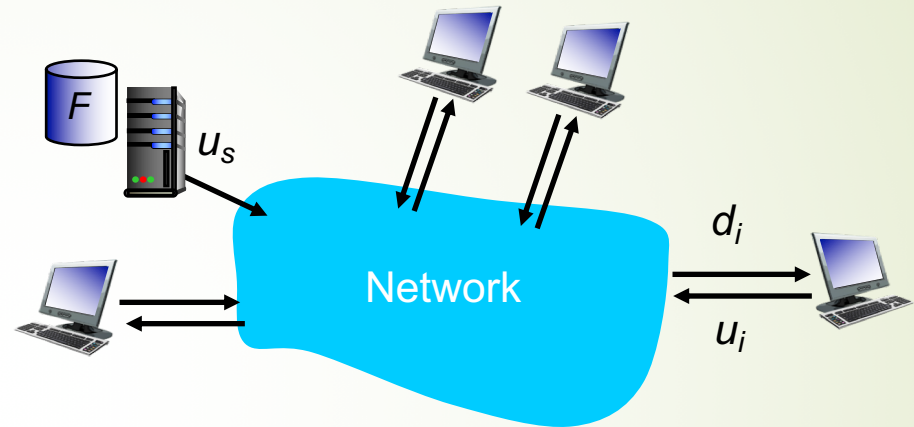
*time to distribute F
to N clients using
client-server approach*

$$D_{c-s} \geq \max\{NF/u_s, F/d_{min}\}$$

increases linearly in N

File Dissemination with P2P

- **server transmission:** must upload at least one copy
 - time to send one copy: F/u_s
- **client:** each client must download file copy
 - min client download time: F/d_{min}
- **clients:** as aggregate must download NF bits
 - max upload rate (limiting max download rate) is $u_s + \sum u_i$



time to distribute F
to N clients using
P2P approach

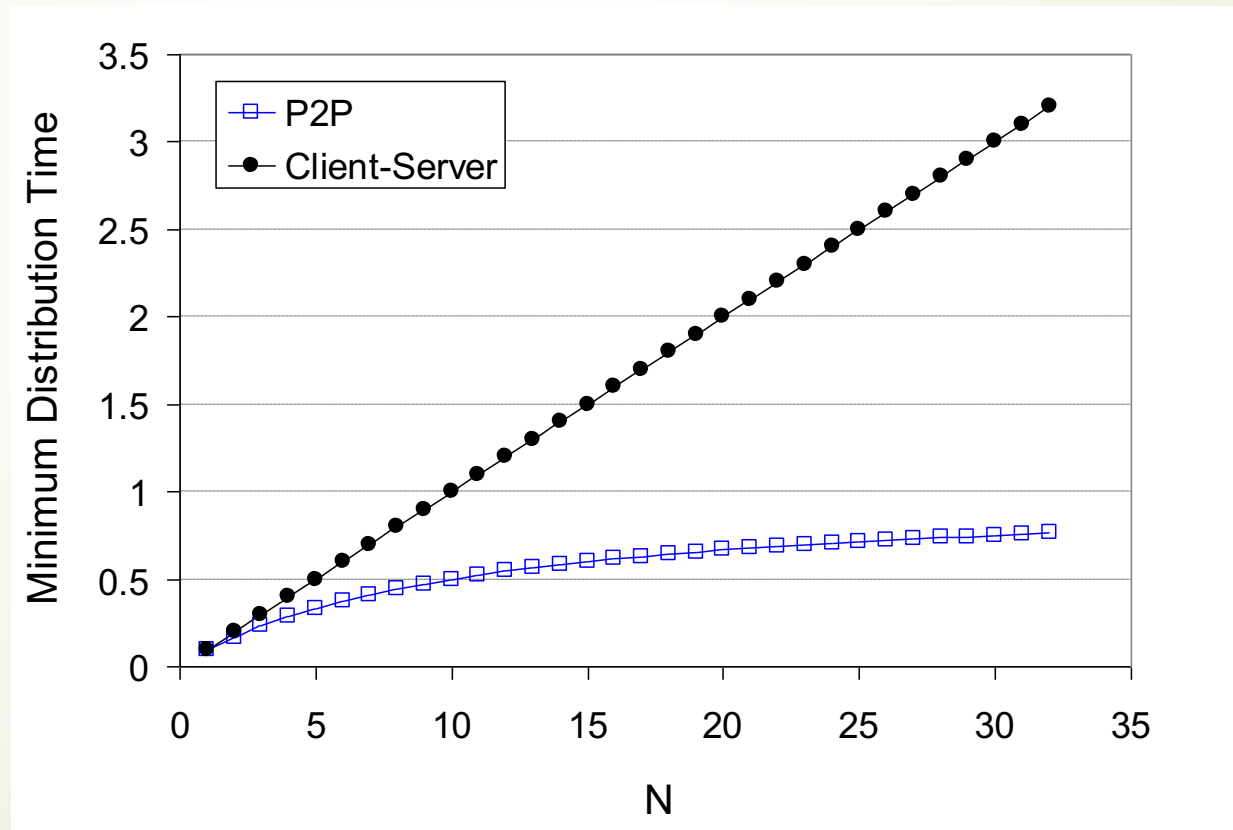
$$D_{P2P} \geq \max\{F/u_s, F/d_{min}, NF/(u_s + \sum u_i)\}$$

increases linearly in N ...

... but so does this, as each peer brings service capacity

Sample Comparison Results: Client-Server versus P2P

client upload rate = u , $F/u = 1$ hour, $u_s = 10u$, $d_{min} \geq u_s$

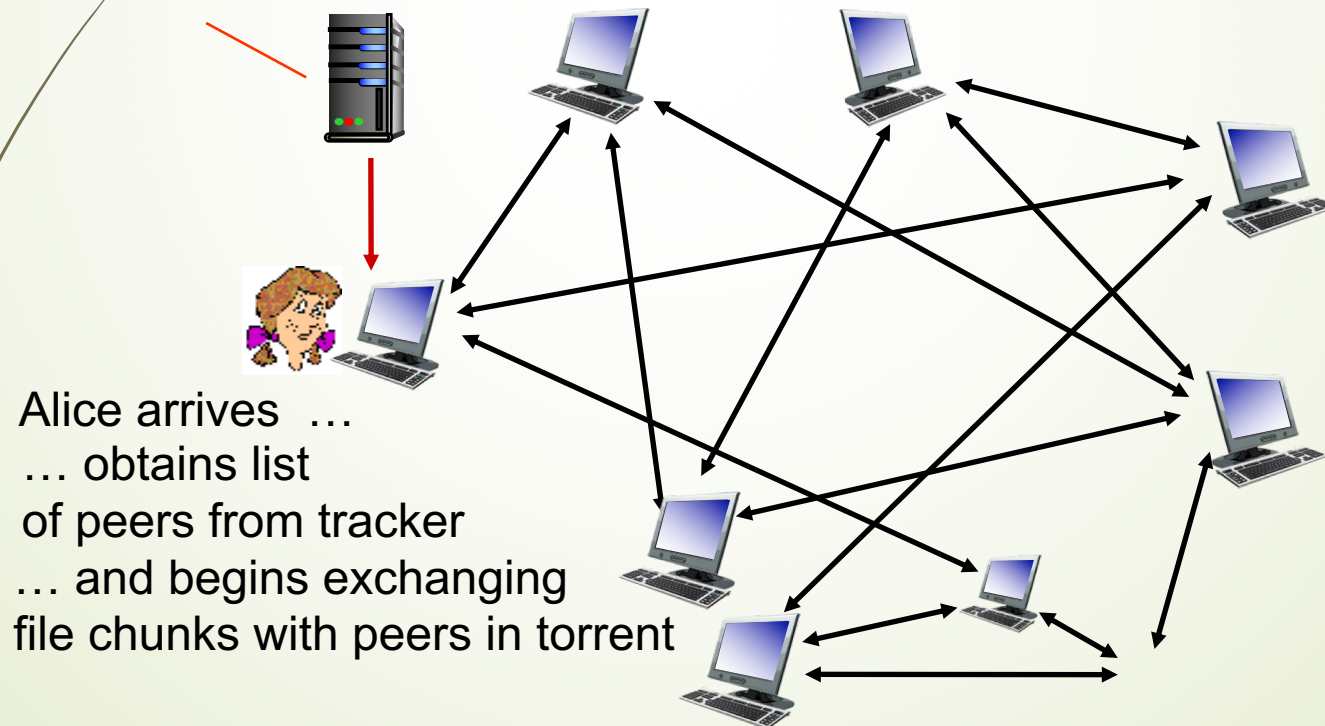


Sample Application: BitTorrent

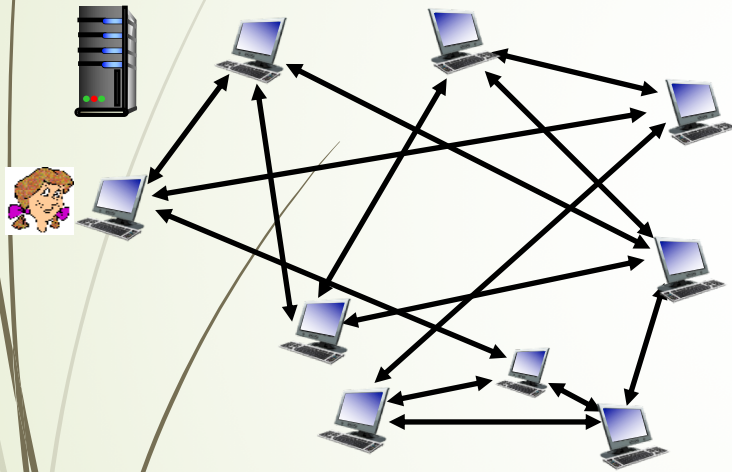
- file divided into 256Kb chunks
- peers in torrent send/receive file chunks

tracker: tracks peers participating in torrent

torrent: group of peers exchanging chunks of a file



Peers Joining/Leaving BitTorrent



- peer joining torrent:
 - has no chunks, but will accumulate them over time from other peers
 - registers with tracker to get list of peers, connects to subset of peers (“neighbors”)
- while downloading, peer uploads chunks to other peers
- peer may change peers with whom it exchanges chunks
- *churn*: peers may come and go
- once peer has entire file, it may (selfishly) leave or (altruistically) remain in torrent

Requesting/Sending Chunks in BitTorrent

requesting chunks:

- at any given time, different peers have different subsets of file chunks
- periodically, Alice asks each peer for list of chunks that they have ← **availability map**
- Alice requests missing chunks from peers, rarest first

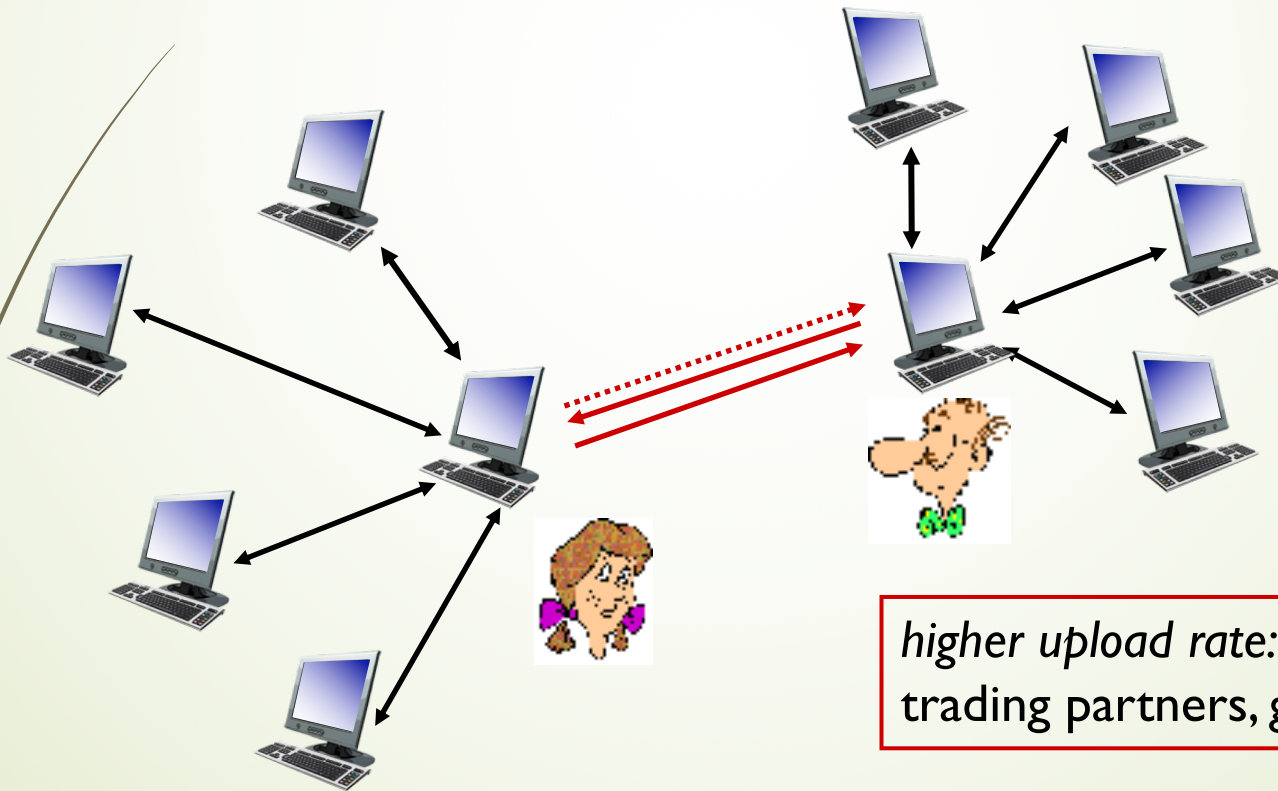
sending chunks: tit-for-tat:

- Alice sends chunks to those four peers currently sending her chunks at highest rate
 - other peers are **choked** by Alice (do not receive chunks from her)
 - re-evaluate top 4 every 10 secs
- every 30 secs: randomly select another peer, starts sending chunks
 - “optimistically unchoke” this peer
 - newly chosen peer may join top 4

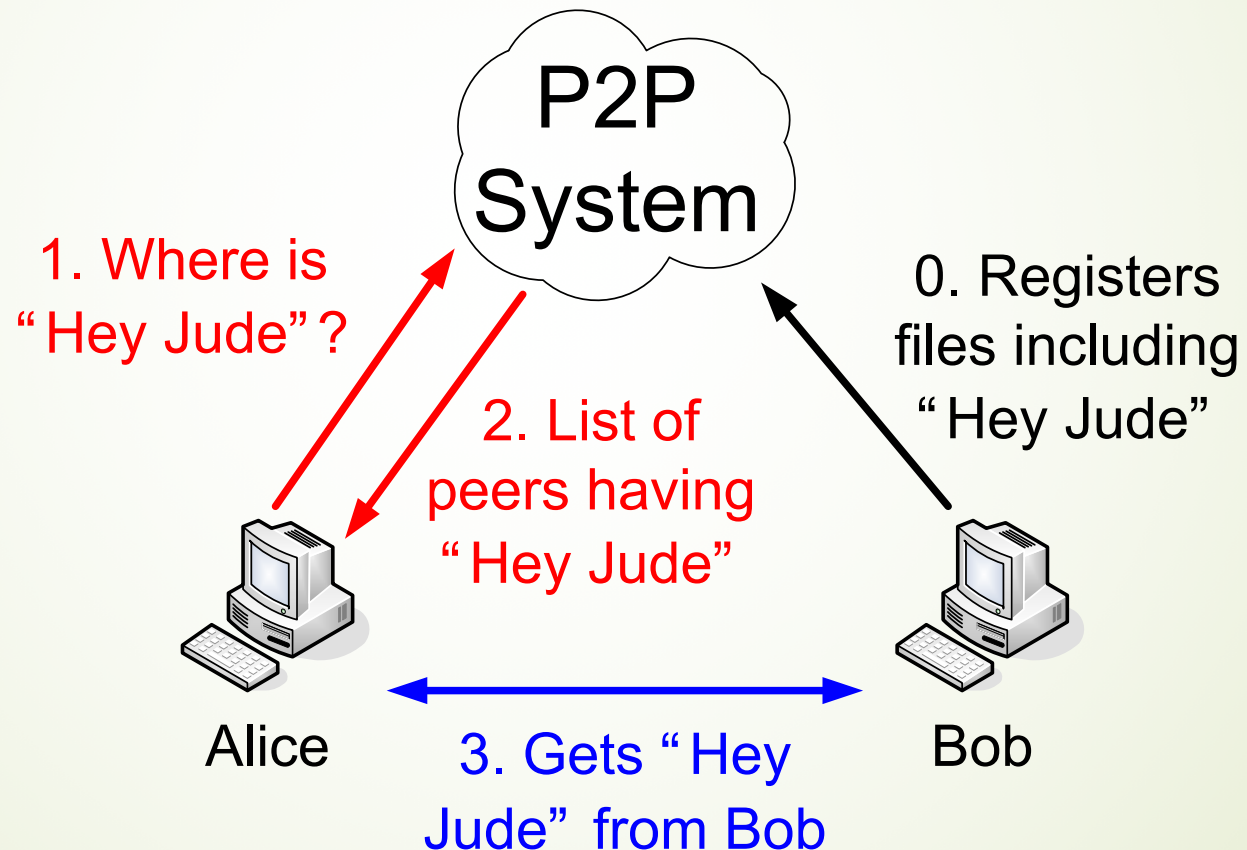
Why do We Need tit-for-tat?

Key of the success of BitTorrent

- (1) Alice “optimistically unchokes” Bob
- (2) Alice becomes one of Bob’s top-four providers; Bob reciprocates
- (3) Bob becomes one of Alice’s top-four providers



Summary: How BitTorrent Work (at Very High Level)



Agenda

- Overview on P2P Networks
- P2P Applications
- Properties of P2P Systems
- Structured and Unstructured P2P
- Distributed Hash Table (DHT)

A Partial List of P2P Applications



Classes of P2P Applications

- Search, File Sharing and Content dissemination
 - Napster, Gnutella, Kazaa, eDonkey, BitTorrent
 - Chord, CAN, Pastry/Tapestry, Kademlia,
 - Bullet, SplitStream, CREW, FareCAST
- Communications
 - MSN, Skype, Social Networking Apps
- Storage
 - OceanStore/POND, CFS (Collaborative FileSystems), TotalRecall, FreeNet, Wuala
- Distributed Computing
 - Seti@home
- **Q: Can you think of anything else?**

P2P/Grid Distributed Processing and Crowdsourcing

- seti@home
 - Search for Extraterrestrial intelligence
 - Central site collects radio telescope data
 - Data is divided into work chunks of 300 Kbytes
 - User obtains client, which runs in background
 - Peer sets up TCP connection to central computer, downloads chunk
 - Peer does FFT on chunk, uploads results, gets new chunk
- Not P2P communication, but exploit **Peer computing power**
- **Crowdsourcing – Human-oriented P2P**

Agenda

- Overview on P2P Networks
- P2P Applications
- Properties of P2P Systems
- Structured and Unstructured P2P
- Distributed Hash Table (DHT)

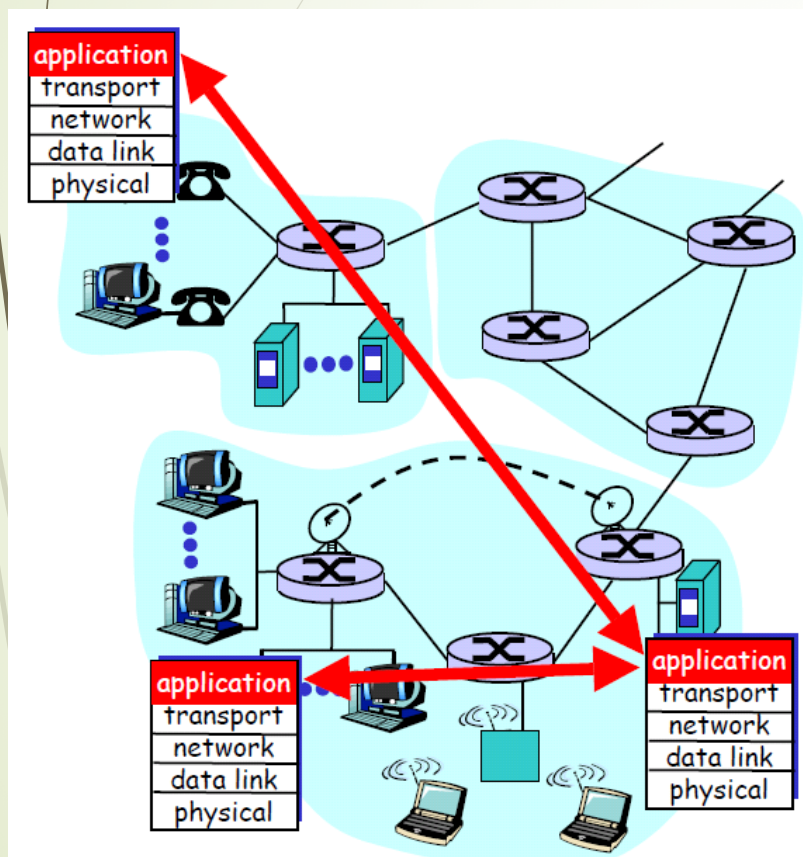
Characteristics of P2P Systems

- Exploit edge resources
 - Storage, content, CPU, Human presence
- Significant autonomy from any centralized authority
 - Each node can act as a Client as well as a Server
- Resources at edge have intermittent connectivity, constantly being added & removed
 - Infrastructure is untrusted and the components are unreliable ← **main weakness....**

Promising Points of P2P

- Self-organizing
- Massive scalability
- Autonomy: non single point of failure
- Resilience to Denial of Service
- Load distribution
- Resistance to censorship

Revisit Overlay Networks



➤ Tremendous design flexibility

- Topology, maintenance
- Message types
- Protocol
- Messaging over TCP or UDP

➤ Underlying physical network is transparent to developer

- But some overlays exploit proximity ← example: reducing RTT among gamers